ForL0: A Cache-Aware Native State Backend for Apache Flink

Author list pending
Huazhong University of Science and Technology and industrial collaborator
Wuhan, China

ABSTRACT

Stateful stream processing increasingly runs with large in-memory working sets, where object-heavy heap layouts, pointer chasing, and garbage collection become first-order costs. We present ForL0, a production-oriented keyed-state backend for Apache Flink that retains the State API and checkpoint semantics while moving the authoritative state into a compact native engine. ForL0 combines key-group and namespace-aware routing, Swiss-table-style group probing, typed JNI paths, an optional set-associative hot-key cache backed by the user-managed L0 partition on Kunpeng processors, and copy-on-write snapshots. It supports ValueState, ListState, MapState, ReducingState, and AggregatingState without application changes.

We evaluate ForL0 on Intel x86_64 and Kunpeng aarch64 servers. All 184 Java and native tests pass, including checkpoint, savepoint, restore, rescale, and mixed-state cases. On Intel, ForL0 improves all 36 WordCount configurations by 1.270× on average and 1.414× at peak; VTune attributes the gain to a reduction in Memory Bound from 48.1% to 15.3%. On NexMark, ForL0 retains a 1.39× geometric-mean speedup even with a normal 20 GB TaskManager budget. Under tight memory, it prevents Full-GC collapse and accelerates Kunpeng q4, q19, and q20 by 4.87×, 5.36×, and 5.96×. These results show that compact state layout and off-heap ownership improve both steady-state efficiency and operational stability, while the L0 cache provides a smaller, workload-dependent increment.

CCS CONCEPTS

• **Information systems** → **Stream management**; *Data structures*; • **Computer systems organization** → *Heterogeneous (hybrid) systems.*

KEYWORDS

stream processing, Apache Flink, state backend, memory locality, native storage, checkpointing

ACM Reference Format:
Author list pending. 2027. ForL0: A Cache-Aware Native State Backend for Apache Flink. In *Proceedings of 2027 ACM SIGMOD/PODS Conference, Industrial Track (SIGMOD '27 Industrial Track).* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Stateful stream processors maintain continuously changing per-key state for aggregation, joins, windows, timers, and online feature computation. Apache Flink exposes this state through a uniform API and couples it with consistent checkpointing and key-group-based redistribution [6, 7]. The abstraction is operationally valuable, but it hides a physical question that becomes decisive at scale: how many objects, pointers, cache lines, and heap-management actions are required for one logical state access?

Flink's heap backend provides a short software path when state is small, but large object graphs increase pointer chasing and garbage-collection pressure. RocksDB bounds JVM heap use, yet introduces serialization, compaction, and storage-engine overhead [11]. This leaves a useful middle ground for latency-sensitive workloads whose state should remain memory-resident: retain Flink's API and fault-tolerance contract, but organize the state as a compact native data structure.

This design problem arose in a joint effort to accelerate Flink Heap State on heterogeneous processors, with a particular interest in Kunpeng's user-managed L0 cache partition. Early prototypes showed that placing a small cache above an object-heavy heap table was insufficient: the main table, JNI boundary, type conversions, and snapshot path still determined most costs. The resulting system, ForL0, therefore treats hardware caching as one component of a complete state backend rather than as an isolated optimization.

ForL0 has three complementary mechanisms. First, it stores authoritative keyed state in native, cache-conscious Swiss-table-style structures partitioned by Flink key-group and namespace. Second, it selects typed JNI paths at state initialization, avoiding repeated type dispatch and eliminating serialization for common primitive and RowData layouts. Third, it takes asynchronous checkpoints from a copy-on-write view, allowing online updates to continue while preserving the checkpoint boundary. On Kunpeng, a set-associative L0 hot-key cache precedes the main table and transparently falls back on a miss.

This paper makes four contributions:

- We describe a complete Flink 1.20.3 keyed-state backend spanning Java API compatibility, typed JNI, a native state engine, checkpoint/restore, and transparent deployment through Flink's SPI.
- We present a locality-oriented state layout and an 8-way L0 hot-key cache whose tags, keys, and values occupy three aligned cache lines per set.
- We show how copy-on-write overlays preserve Flink checkpoint, savepoint, and rescale semantics without maintaining a duplicate Java state table.
- We report a two-platform evaluation covering 184 correctness tests, 36-point WordCount grids, 14 state operations, eight NexMark queries, three memory budgets, an L0 ablation, and VTune microarchitectural evidence.

The central empirical result is not limited to memory exhaustion. Under a normal 20 GB TaskManager budget, ForL0 still reduces the aggregate runtime of eight NexMark queries from 509 s to 343 s and achieves a 1.39× geometric-mean speedup. Tight memory amplifies the benefit by exposing the instability of heap-resident state, but it is not the only condition under which the backend helps.

2 CONTEXT AND REQUIREMENTS

2.1 Flink Keyed State

Flink partitions keyed state into key-groups. Each state descriptor supplies key, namespace, and value serializers, while the backend implements the physical representation and participates in checkpoint, savepoint, restore, and rescale. Operators may use a void namespace, as in simple per-key aggregates, or many namespaces, as in windowed workloads. A replacement backend must preserve these semantics across ValueState, ListState, MapState, ReducingState, and AggregatingState.

The backend must also coexist with Flink-managed priority queues and timers. ForL0 deliberately reuses Flink's priority-queue implementation and replaces only the primary keyed-state data plane. This limits semantic risk and isolates the part of the runtime where memory layout is most important.

2.2 Why the Heap Path Becomes Expensive

Heap-resident state compounds three costs. First, hash metadata, boxed keys, namespaces, and values are separate objects, so one lookup may require several dependent cache misses. Second, a large live object graph increases tracing and evacuation work for the JVM. Third, generic serializers and state wrappers repeat type checks on the per-record path. These costs interact: under generous memory, they reduce throughput; under tight memory, they can trigger repeated Full GC and turn a slow query into an operational failure.

Our design requirements are therefore: (1) preserve Flink semantics and deployment compatibility; (2) reduce pointer chasing and heap residency; (3) specialize common layouts without losing a generic fallback; (4) keep checkpoint preparation short and serialization asynchronous; and (5) exploit Kunpeng L0 when present without making correctness depend on it.

3 SYSTEM ARCHITECTURE

Figure 1 shows the four-layer organization. The Java control layer implements Flink interfaces, tracks the current key and key-group, validates serializers, registers state, and precomputes access strategies. Typed JNI functions carry compact arguments into the native engine. The native engine owns StateTables, namespace routing, Swiss tables, hot-key caches, and copy-on-write metadata. It is the single source of truth for keyed state; Java does not maintain a shadow copy.

3.1 State Registration and Access Strategies

When a state descriptor is first used, the Java layer validates serializer compatibility and maps it to a native type layout. Supported primitive layouts include 32- and 64-bit integers, floating-point values, and booleans; structured layouts include strings, byte arrays, lists, maps, fixed rows, void namespaces, and TimeWindow namespaces. The backend then selects a ValueState, ListState, or MapState strategy once. Subsequent accesses enter a narrow JNI function instead of repeating serializer inspection.

RowData keys use three paths: a single primitive field is unboxed directly, fixed-length multi-field rows map to a compact native FixedRow, and variable-length rows use a generic byte representation. Selected BinaryRowData values can be returned through a read-only native memory view, avoiding an intermediate byte-array copy. Unsupported combinations retain a serializer-mediated fallback, so specialization does not narrow the State API.

3.2 State Ownership and Namespace Routing

Each task owns one StateEngine. A registered descriptor maps to one typed StateTable, and the table is partitioned first by key-group and then by namespace:

$$S : \text{keyGroup} \rightarrow \text{namespace} \rightarrow \text{SwissTable}\langle K, V\rangle. \tag{1}$$

For a void namespace, a key-group points directly to its SwissTable and avoids a namespace lookup. General namespaces use a map from namespace to independent sub-tables; empty namespace tables are removed eagerly. This layout aligns storage, checkpoint iteration, and rescale granularity.

Table 1 makes the per-state mapping explicit. The design does not encode a Java object graph in native memory. Instead, the outer keyed-state lookup and, where applicable, the inner container use independently typed layouts. ReducingState and AggregatingState deliberately keep user-code execution in Java: only the durable accumulator crosses into the native data plane.

4 CACHE-AWARE STATE ACCESS

4.1 Swiss-Table Main Store

The authoritative table stores one control byte per slot in a contiguous control array and key/value payloads inline. A hash is split into H_1, which selects the first probe group, and a seven-bit H_2 fingerprint stored in the control byte:

$$H_1 = h \gg 7, \qquad H_2 = h \,\&\, 0x7F. \tag{2}$$

A probe loads a group of control bytes, compares all fingerprints with SWAR or architecture-specific SIMD, and dereferences only matching payload slots. Empty and deleted markers support early termination and tombstones. Capacity is a power of two; growth performs a rehash into a larger table. Unlike a separate index/value-store design, the selected key and value are colocated in the slot, avoiding an additional value-pointer traversal.

4.2 Kunpeng L0 Hot-Key Cache

On supported Kunpeng systems, ForL0 allocates an outer cache from the user-managed L0 partition. It is a set-associative hash table with N sets and eight ways per set. Each aligned set occupies 192 bytes: one 64-byte tag row, one key row, and one value row. A mixed 64-bit hash supplies H_2 as the tag and $H_1 \,\&\, (N-1)$ as the set index. NEON `vceq_u8` compares all eight tags; a scalar path is used elsewhere. A hit reads the 64-byte tag row plus one 8-byte key and value, approximately 80 bytes total.

Updates are write-through: the SwissTable is updated first, then the cache entry is inserted or replaced. Empty ways are preferred; otherwise a three-bit round-robin pointer stored in the tag row selects a victim. Deletes invalidate the tag. The cache is never part of the checkpoint image and can be rebuilt after restore, which keeps it outside the correctness boundary. If the L0 device or runtime

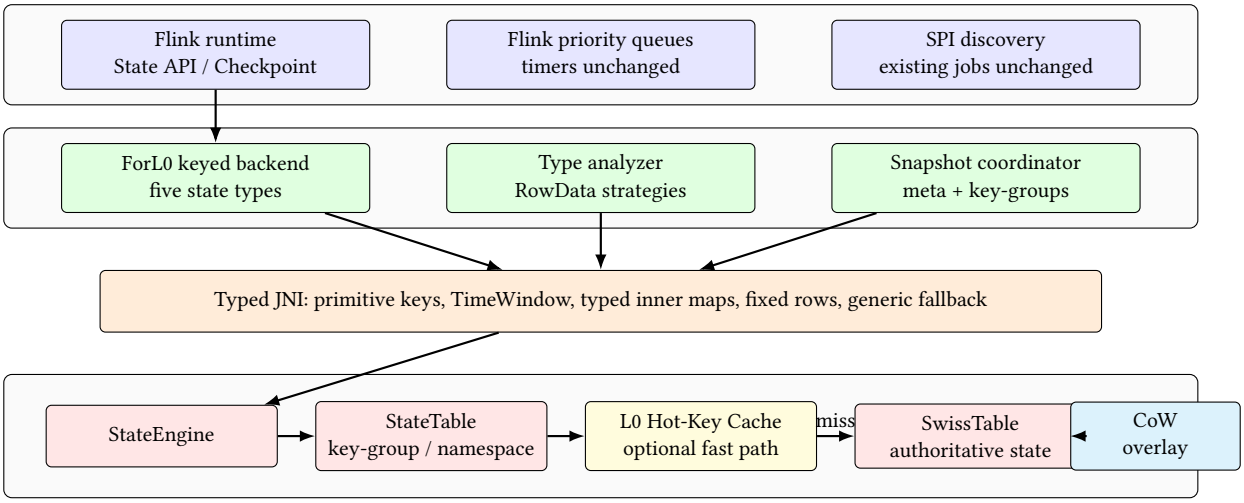


Figure 1: ForL0 architecture. Flink-facing control remains in Java; the authoritative keyed-state data plane, optional L0 cache, and snapshot overlay reside in the native engine.

Table 1: State API to native representation and execution path. "Typed" means that serializer inspection is performed at registration rather than on every access.

Flink state	Native value	Common operation path	Compatibility / fallback
ValueState	Scalar, bytes, string, or fixed row	Typed point lookup; primitive return or read-only BinaryRowData view when eligible	Generic serializer path for variable layouts
ListState	Native element vector	Append/update in native container; batch iterator amortizes JNI on reads	Serializer-mediated element path; namespace merge preserves Flink order
MapState	Typed inner hash map	Outer key routes once; inner get/put/remove/contains avoids materializing the full Java map	Opaque serialized map for unsupported key/value combinations
ReducingState	Reduced value	Read accumulator, execute `ReduceFunction` in Java, write typed result	User function and null semantics remain unchanged
AggregatingState	Accumulator	Native accumulator storage; create/add/merge/result executed by the Flink function	Result materialization follows the registered serializer

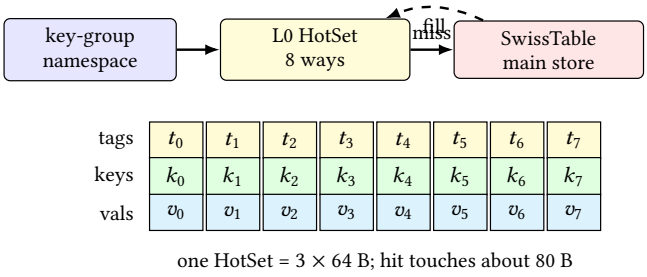


Figure 2: Two-level lookup and the 8-way L0 HotSet. Tags, keys, and values occupy three aligned 64-byte rows; a miss falls back to the authoritative SwissTable.

library is absent, allocation transparently falls back and the main table remains fully functional.

4.3 Five State Types

ValueState maps to point lookup, update, and removal. ListState stores a native list container and supports append, update, and iteration. MapState uses a typed inner map so that an outer Flink key routes to a native map without reconstructing a Java object graph. ReducingState and AggregatingState invoke the user function in Java but store the accumulated value through the same typed native path. The common StateTable substrate ensures that all five types share namespace isolation, key-group ownership, snapshots, and restore.

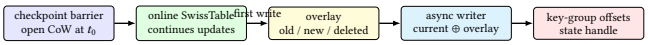


Figure 3: Copy-on-write checkpoint path. The synchronous phase establishes the t_0 boundary; serialization proceeds asynchronously while the task continues processing.

Table 2: Copy-on-write transition and asynchronous output rule.

First action after t_0	Overlay record	Snapshot output
Update	old value	old value replaces current value
Insert	post-t_0 marker	skip current entry
Delete	old value + deleted flag	emit old value from overlay
No change	none	emit current table value

5 CHECKPOINT, RESTORE, AND RESCALE

Flink initiates a checkpoint on the task thread but writes state asynchronously. ForL0 separates these phases. During synchronous preparation, the StateEngine marks every participating key-group as snapshot-active and creates an overlay. The online table remains writable. On the first update after the boundary, the overlay records the old value; a new key is marked as post-snapshot and excluded; a deletion records the deleted value. The asynchronous writer reconstructs the t_0 view by merging the current table with the overlay.

The writer emits Flink's keyed-backend serialization metadata, priority-queue blocks, and native KV blocks with state identifiers and entry counts. Namespace, key, and value encodings follow their serializers. Restore validates the key serializer, recreates state metadata, restores priority queues, then rebuilds typed StateTables key-group by key-group. Canonical savepoints use the same builder with a translation step. Rescale redistributes key-groups; L0 sets are discarded and lazily repopulated.

The snapshot merge obeys three cases for every key first touched after t_0. An update records the pre-update value exactly once; an insertion records a post-snapshot marker; and a deletion retains the pre-delete value. During asynchronous enumeration, current entries marked as post-snapshot are skipped, current values with an overlay are replaced by the recorded old value, and deleted old values are emitted from the overlay. Therefore each key contributes exactly the value visible at t_0, independent of the number of subsequent writes.

Snapshot files are organized by key-group rather than by native allocation. Each key-group block carries the registered state identifier, entry count, namespace, key, and value bytes after serializer-compatible encoding. This is the unit Flink already assigns during rescale: restore reads only the handles intersecting the target key-group range and inserts entries into the newly constructed typed tables. The native memory address, SwissTable capacity, probe order, and hot-cache contents are intentionally absent from the durable format.

Resource lifetime is explicit. Successful and failed snapshots both release overlay state, restore failures destroy partially created engines, and backend disposal releases native tables and caches. This was essential in containerized deployments, where a leaked engine survives longer than an individual operator attempt.

6 IMPLEMENTATION AND DEPLOYMENT

ForL0 targets Flink 1.20.3 and consists of a Java backend JAR plus a native shared library. The Java classes implement the standard backend factory and are discovered through SPI, so an existing job selects ForL0 through configuration without changing application code. Native code uses C++17 and compiles on x86_64 and aarch64. The Kunpeng build enables NEON matching and loads the L0 memory-pool library dynamically; L0 is a runtime option rather than a build-time dependency.

The JNI boundary carries engine and state handles, key-group identifiers, and typed payloads. One logical state operation performs one JNI transition in the common path. Batch and iterator interfaces amortize transitions for list and map traversal. The deployment bundle includes container images, offline dependencies, WordCount and NexMark harnesses, and build, usage, design, and test documents. This packaging matters for an industrial backend: reproducible installation and transparent fallback are as important as an isolated data-structure speedup.

7 EVALUATION

We ask six questions: (Q1) Does ForL0 preserve Flink state semantics? (Q2) Does the layout improve isolated state operations? (Q3) Does it improve a controlled per-record state workload? (Q4) Does it help end-to-end queries under normal memory? (Q5) Does off-heap state prevent collapse under tight memory? (Q6) What portion comes from L0 versus the main native layout?

7.1 Platforms, Baselines, and Workloads

Table 3 summarizes the environments. HashMapStateBackend is the direct in-memory baseline. WordCount performs a ValueState read-modify-write for two billion records at 36 combinations of 1–10 million keys and skew 0–1.0. The official Flink StateBackend JMH suite isolates ValueState, ListState, and MapState operations. NexMark uses q4, q5, q8, q9, q11, q18, q19, and q20, covering aggregation, windows, joins, lists, and maps [4].

Measurement protocol. The WordCount grid contains $6 \times 6 = 36$ configurations and each backend processes exactly 2×10^9 records per configuration with checkpointing disabled. Throughput is the sink-observed total over parallelism eight. NexMark reports end-to-end completion time for the same query, event count, topology, and container limits under each backend. Ratios use T_{heap}/T_{ForL0}, so values above one favor ForL0; the cross-query summary is the geometric mean of per-query ratios rather than a ratio of unlike event totals. The available campaign records one completed run per configuration and therefore supports controlled case comparisons, not confidence intervals; repeated-run variance is a remaining measurement task.

Table 3: Experimental platforms and workload controls.

Item	Configuration	Purpose
Software	Flink 1.20.3; Java 8+; containerized standalone cluster; parallelism 8	Same application logic and state API for both backends
Intel x86_64	1 JobManager + 2 TaskManagers; 4 cores/TM for the default run; 16/18/20 GB TM memory with 12/14/15 GB heap; no L0 hardware	WordCount, JMH, NexMark memory sweep, VTune
Kunpeng aarch64	openEuler; 4-core TM limit; 11.7 GB heap; 512 MB managed memory; L0 on/off	Tight-memory NexMark and L0 ablation
WordCount	2×10^9 records/run; keys $\in \{1, 2, 4, 6, 8, 10\}$M; skew $\in \{0, .2, .4, .6, .8, 1\}$; checkpoint off	Controlled steady-state state-access cost
NexMark	q4, q5, q8, q9, q11, q18, q19, q20	End-to-end stateful SQL/DataStream behavior

Table 4: NexMark query mix. Event counts differ by query so that each run reaches a stable, state-intensive phase while remaining operationally tractable.

Query	Events	Stateful pattern	Query	Events	Stateful pattern
q4	80M	windowed category average	q5	80M	sliding-window hot items
q8	100M	session window / new users	q9	40M	two-stream winning-bid join
q11	80M	per-user session window	q18	80M	latest-result deduplication
q19	80M	per-auction Top-10	q20	60M	grouped price aggregation

Table 5: Correctness coverage; all listed tests pass.

Layer	Tests	Representative coverage
Java / Flink	54	Five state types; checkpoint/savepoint; async snapshot; restore; rescale; windows; SPI
Native engine	130	SwissTable; CoW; serialization round trip; namespaces; typed maps; HotCache
Total	184	184 passed, 0 failed

Table 6: Selected Intel StateBackend JMH changes over heap.

Operation	Change	Operation	Change
value.Add	+87.2%	value.Update	+59.5%
map.Add	+110.0%	map.Remove	+74.4%
list.Iterate	+131.5%	list.Append	−8.9%

7.2 Q1: Correctness and Compatibility

All 184 tests pass: 54 Java integration/unit tests and 130 native tests. Table 5 shows the coverage rather than only the total. The campaign exercises all five state types, namespace semantics, periodic and externalized checkpoints, savepoints, asynchronous updates during a snapshot, restore, rescale, window aggregation, skew and stress, L0 fallback, and SPI discovery. Native tests cover SwissTable growth and tombstones, CoW round trips, serialization, TimeWindow namespaces, typed inner maps, and cache lifecycle.

The result establishes semantic compatibility, not checkpoint-speed superiority. The current campaign validates completion and exact restored values; a matched checkpoint-duration and recovery-time study remains future work.

7.3 Q2: Operator-Level State Cost

On Intel, 13 of 14 JMH operations improve. The largest gains occur for list iteration (+131.5%), map insertion (+110.0%), value insertion (+87.2%), map removal (+74.4%), and value update (+59.5%). List append regresses by 8.9%, because a single append performs little probing work and exposes container/JNI overhead. The result is useful as a boundary: ForL0 helps most when an operation performs lookup, iteration, or update work that benefits from compact layout; it is not uniformly faster for trivial calls.

7.4 Q3: WordCount and Microarchitecture

Table 7 reports representative points from the Intel grid. ForL0 wins all 36 combinations, with a mean of 1.270× and a peak of 1.414×. Each run processes two billion records; the mean wall-clock reduction is about 37 s. Intel has no L0 facility, so this result isolates the native layout, specialized access path, and group probing rather than the hardware cache.

VTune explains the gain. Across matched 60.001 s collection windows, Memory Bound falls from 48.1% to 15.3%, DRAM Bound from 30.8% to 2.2%, and Back-End Bound from 58.1% to 29.5%. CPI decreases from 0.615 to 0.375, while Retiring rises from 30.4% to

Table 7: Representative Intel WordCount points (M records/s). All 36 scanned points favor ForL0.

Keys	Skew	Heap	ForL0	Ratio	Keys	Skew	Heap	ForL0	Ratio
1M	0.0	13.153	17.464	1.328	2M	0.0	12.140	17.170	1.414
4M	0.4	9.657	12.858	1.332	6M	0.8	10.556	13.297	1.260
8M	1.0	11.162	15.028	1.346	10M	0.2	9.933	12.476	1.256
10M	1.0	12.616	14.855	1.178			Mean over 36 points		1.270

Table 8: Intel NexMark with normal memory (20 GB TM / 15 GB heap), seconds.

	q4	q5	q8	q9	q11	q18	q19	q20
Heap	57	64	38	45	37	61	123	84
ForL0	57	52	36	45	37	35	43	38
Ratio	1.00	1.23	1.06	1.00	1.00	1.74	2.86	2.21

Table 9: Kunpeng NexMark under tight memory, seconds. Stars mark heap runs heavily delayed by Full GC.

Backend	q4	q5	q8	q9	q11	q18	q19	q20
Heap	716★	103	72	201★	98	134	638★	542★
ForL0 with L0	147	103	62	105	96	93	119	91
ForL0 without L0	159	100	62	107	102	98	121	94
Heap / ForL0	4.87	1.00	1.16	1.91	1.02	1.44	5.36	5.96

50.9%. The reduction in memory stalls is therefore visible both in counters and application throughput.

Kunpeng WordCount is a contrasting result. Its 36-point mean ratio is 0.96×, with ratios concentrated between 0.88 and 1.03. The workload performs only one read-modify-write per record and creates little GC pressure, so the fixed JNI cost offsets the native lookup benefit on this aarch64 system. This negative result prevents us from attributing all cross-platform gains to the hash table alone.

7.5 Q4: NexMark under Normal Memory

Table 8 reports the Intel 20 GB setting, which provides a 15 GB heap and avoids the most severe memory pressure. ForL0 still improves five of eight queries and matches heap on the other three. Across the eight queries, total runtime falls from 509 s to 343 s (32.6%), and the geometric-mean speedup is 1.39×. q19 and q20 improve by 2.86× and 2.21×, while q4, q9, and q11 remain at parity. Thus, normal-memory NexMark already shows a material advantage; tight memory magnifies rather than creates it.

7.6 Q5: Tight-Memory Stability

On Kunpeng with 11.7 GB heap, 512 MB managed memory, and four TM cores, heap state repeatedly enters Full GC on q4, q9, q19, and q20. Table 9 shows that ForL0 completes every query. q4 drops from 716 s to 147 s (4.87×), q19 from 638 s to 119 s (5.36×), and q20 from 542 s to 91 s (5.96×). q5 and q11 remain near parity, showing that gains follow state and memory pressure rather than a global runtime bias.

The Intel memory sweep exhibits the same transition. At 16 GB, heap q4 and q9 take 352 s and 232 s, while q19 and q20 fail to complete; ForL0 finishes them in 61, 46, 47, and 40 s. At 18 GB, heap q19 and q20 still require 279 and 313 s versus 45 and 38 s. At 20 GB, the gap narrows but remains significant for q18–q20. Native ownership therefore converts a steep, GC-driven performance cliff into a comparatively flat response to the memory budget.

7.7 Q6: L0 Contribution

The Kunpeng ablation in Table 9 shows a modest L0 increment over the native main table: q4 improves from 159 s to 147 s, q18 from 98 s to 93 s, and q20 from 94 s to 91 s; several queries are unchanged within a few seconds. NexMark performs substantial serialization, window management, and data exchange, so many records do not repeatedly hit a small value-state working set. The dominant end-to-end benefit is the native layout and removal of large state from the Java heap. L0 is most useful for fine-grained hot-key access and should be described as an accelerator, not as the sole cause of the system-level result.

8 INDUSTRIAL LESSONS AND LIMITATIONS

A cache is not a backend. The initial goal emphasized Kunpeng L0, but experiments forced a broader design. Caching cannot repair pointer-heavy authoritative storage, repeated serialization, or a stop-the-world heap. The main contribution became an end-to-end native backend; L0 then added value on top of a sound data plane.

Normal and tight memory are different claims. Under normal memory, compact layout and typed paths improve state-heavy queries without relying on GC collapse. Under tight memory, moving state off heap additionally prevents Full GC and non-completion. Reporting only the largest tight-memory speedups would hide this distinction and overstate the role of one mechanism.

Fallbacks enabled deployment. The generic serializer path, scalar group match, and non-L0 allocator made the same plugin usable on Intel and Kunpeng. They also allowed correctness testing without special hardware. This widened the system's practical scope, although the fastest path still depends on recognizable primitive or fixed-row layouts.

Limitations. The two platforms are not intended as a normalized CPU comparison; they answer complementary questions under their own controlled baselines. The L0 ablation is end-to-end and does not include a dedicated hit-rate or cycle-level microbenchmark. Checkpoint and restore are validated for correctness, but matched duration and recovery-time numbers are

Table 10: Intel NexMark memory sweep, seconds. "–" denotes a heap run that did not complete because of repeated Full GC. H is JVM heap within the TaskManager budget.

TM / heap	Backend	q4	q5	q8	q9	q11	q18	q19	q20
16G / H12G	Heap	352	63	39	232	44	53	–	–
	ForL0	61	56	36	46	35	34	47	40
18G / H14G	Heap	156	64	36	46	53	53	279	313
	ForL0	57	52	36	46	37	38	45	38
20G / H15G	Heap	57	64	38	45	37	61	123	84
	ForL0	57	52	36	45	37	35	43	38

not yet reported. The evaluation focuses on in-memory HashMapStateBackend rather than RocksDB/ForSt, and does not yet cover timer-dominated applications or long-running production traces. Finally, ListState append exposes JNI/container overhead and remains a target for batching.

9 RELATED WORK

Flink, MillWheel, Naiad, the Dataflow model, and Structured Streaming establish the semantics and runtime models for consistent stateful stream processing [2, 3, 5, 7, 10]. Flink's state-management work focuses on checkpoint consistency and operational integration [6], while Megaphone addresses low-disruption state migration [9]. ForL0 preserves those semantic boundaries but focuses on the physical representation and access cost of local keyed state.

RocksDB provides a mature persistent backend with bounded heap use, compaction, and disk-aware behavior [11]. ForL0 targets a different point: memory-resident workloads that need low latency and predictable behavior without a Java object graph. FASTER demonstrates that carefully designed in-place updates and memory tiers can substantially change key-value performance [8]. Swiss tables demonstrate the value of compact control metadata and group probing [1]; ForL0 adapts these ideas to Flink key-groups, namespaces, typed state containers, checkpoints, and an optional hardware-resident hot-key cache.

10 CONCLUSION

ForL0 is a complete native keyed-state backend for Apache Flink, not merely a hash-table or cache prototype. It integrates five state types, typed JNI and RowData paths, cache-aware Swiss tables, an optional Kunpeng L0 hot-key cache, and copy-on-write checkpoint/restore behind the standard State API. All 184 correctness tests pass. The evaluation shows 1.270× mean WordCount throughput on Intel, reduced memory stalls, a 1.39× geometric-mean NexMark speedup under normal memory, and up to 5.96× under tight memory where heap state suffers Full GC. The results also establish a useful boundary: L0 adds a workload-dependent increment, while compact native state ownership provides the broad performance and stability benefit.

REFERENCES

[1] Abseil. 2018. Swiss Tables Design Notes. https://abseil.io/about/design/swisstables.

[2] Tyler Akidau, Alex Balikov, Kaya Bekiroğlu, Slava Chernyak, Josh Haberman, Reuven Lax, Sam McVeety, Daniel Mills, Paul Nordstrom, and Sam Whittle. 2013. MillWheel: Fault-Tolerant Stream Processing at Internet Scale. *Proceedings of the VLDB Endowment* 6, 11 (2013), 1033–1044.

[3] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, and Sam Whittle. 2015. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1792–1803.

[4] Apache Beam Community. 2024. NexMark Benchmark Suite. https://beam.apache.org/documentation/sdks/java/testing/nexmark/. Accessed 2026-04-24.

[5] Michael Armbrust, Tathagata Das, Joseph Torres, Burak Yavuz, Shixiong Zhu, Reynold Xin, Ali Ghodsi, Ion Stoica, and Matei Zaharia. 2018. Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark. In *Proceedings of the 2018 International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 601–613.

[6] Paris Carbone, Stephan Ewen, Gyula Fóra, Seif Haridi, Stefan Richter, and Kostas Tzoumas. 2017. State Management in Apache Flink®: Consistent Stateful Distributed Stream Processing. *Proceedings of the VLDB Endowment* 10, 12 (2017), 1718–1729.

[7] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. 2015. Apache Flink: Stream and Batch Processing in a Single Engine. *IEEE Data Engineering Bulletin* 38, 4 (2015), 28–38.

[8] Badrish Chandramouli, Guna Prasaad, Donald Kossmann, Justin Levandoski, James Hunter, and Mike Barnett. 2018. FASTER: A Concurrent Key-Value Store with In-Place Updates. In *Proceedings of the 2018 International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 275–290.

[9] Hyeontaek Lim Hoffman, Ryan Stutsman, Frank McSherry, and Malte Schwarzkopf. 2019. Megaphone: Latency-Conscious State Migration for Distributed Streaming Dataflows. In *Proceedings of the 2019 International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 435–450.

[10] Derek G. Murray, Frank McSherry, Rebecca Isaacs, Michael Isard, Paul Barham, and Martín Abadi. 2013. Naiad: A Timely Dataflow System. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 439–455.

[11] RocksDB Project. 2024. RocksDB. https://rocksdb.org/. Accessed 2026-04-24.